

WEEK 10 PRACTICAL

QUESTION-1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

AIM

To investigate inode allocation and examine the difference between hard links and symbolic links using Linux file-system commands.

PROGRAM

```
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ echo "OSSP Week 10 Inode Experiment" > inode_file.txt
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ ln inode_file.txt hard_inode.txt
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ ln -s inode_file.txt soft_inode.txt
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ ls -li inode_file.txt hard_inode.txt soft_inode.txt
3940649675029981 hard_inode.txt 3940649675029981 inode_file.txt 844424931212773 soft_inode.txt

ashborn@DESKTOP-I4G5Q4N:~/week10_p$ stat inode_file.txt hard_inode.txt soft_inode.txt
  File: inode_file.txt
  Size: 30             Blocks: 0          IO Block: 512    regular file
Device: 0,2    Inode: 3940649675029981  Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/ ashborn)   Gid: ( 1000/ ashborn)
Access: 2026-09-28 05:48:18.261051600 +0000
Modify: 2026-09-28 05:48:18.261051600 +0000
Change: 2026-09-28 05:48:18.304081000 +0000
 Birth: -
  File: hard_inode.txt
  Size: 30             Blocks: 0          IO Block: 512    regular file
Device: 0,2    Inode: 3940649675029981  Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/ ashborn)   Gid: ( 1000/ ashborn)
Access: 2026-09-28 05:48:18.261051600 +0000
Modify: 2026-09-28 05:48:18.261051600 +0000
Change: 2026-09-28 05:48:18.304081000 +0000
 Birth: -
  File: soft_inode.txt -> inode_file.txt
  Size: 14             Blocks: 0          IO Block: 512    symbolic link
Device: 0,2    Inode: 844424931212773  Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/ ashborn)   Gid: ( 1000/ ashborn)
Access: 2026-09-28 05:48:19.944843500 +0000
Modify: 2026-09-28 05:48:19.944843500 +0000
Change: 2026-09-28 05:48:19.944843500 +0000
 Birth: -
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ find . -maxdepth 1 -inum "$(stat -c '%i' inode_file.txt)"
./hard_inode.txt
./inode_file.txt
```

Commands –

```
echo "OSSP Week 10 Inode Experiment" > inode_file.txt ln inode_file.txt
hard_inode.txt ln -s inode_file.txt soft_inode.txt ls -li inode_file.txt
hard_inode.txt soft_inode.txt stat inode_file.txt hard_inode.txt soft_inode.txt
find . -maxdepth 1 -inum "$(stat -c '%i' inode_file.txt)"
```

OUTPUT

The output will show that the original file and hard link have the same inode number, while the symbolic link has its own different inode number.

The stat command will also show inode information and the increased link count for the original file/hard link. The find command will locate both files sharing the same inode.

OBSERVATION

The experiment demonstrates that a hard link points directly to the same inode as the original file, so both names refer to the same file data. A symbolic link has a separate inode and stores a reference to the original file.

RESULT

The inode structure, hard link, and symbolic link were successfully created and examined using ls -li, stat, and find.

QUESTION-2: Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read()/write() operations in terms of efficiency and implementation complexity.

AIM

To implement file reading and writing using mmap() and compare it with traditional read()/write() operations.

PROGRAM

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <time.h>
#include <string.h>

#define FILE_NAME "mmap_test.txt"
#define SIZE (1024 * 1024) // 1 MB
#define REPEAT 50

double get_time() {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main() {
    int fd = open(FILE_NAME, O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    if (ftruncate(fd, SIZE) == -1) {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    char *buffer = malloc(SIZE);
    if (!buffer) {
        perror("malloc");
        close(fd);
        return 1;
    }

    memset(buffer, 'A', SIZE);

    /* Traditional read/write */
    double start = get_time();

    for (int i = 0; i < REPEAT; i++) {
        lseek(fd, 0, SEEK_SET);
        write(fd, buffer, SIZE);
    }
}

```

```

        write(fd, buffer, SIZE);

        lseek(fd, 0, SEEK_SET);
        read(fd, buffer, SIZE);
    }

    double traditional_time = get_time() - start;

    /* Memory-mapped I/O */
    char *mapped = mmap(NULL, SIZE, PROT_READ | PROT_WRITE,
                        MAP_SHARED, fd, 0);

    if (mapped == MAP_FAILED) {
        perror("mmap");
        free(buffer);
        close(fd);
        return 1;
    }

    start = get_time();

    for (int i = 0; i < REPEAT; i++) {
        memcpy(mapped, buffer, SIZE);

        volatile char value = mapped[0];
        (void)value;
    }

    msync(mapped, SIZE, MS_SYNC);

    double mmap_time = get_time() - start;

    printf("File size          : 1 MB\n");
    printf("Repetitions           : %d\n\n", REPEAT);

    printf("Traditional I/O   : %.6f seconds\n", traditional_time);
    printf("Memory-mapped I/O: %.6f seconds\n\n", mmap_time);

    printf("Memory-mapped I/O avoids repeated read()/write() system calls\n");
    printf("and provides direct access to the file through memory.\n");

    munmap(mapped, SIZE);
    free(buffer);
    close(fd);

    return 0;
}

```

OUTPUT

```
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ nano q2.c
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ gcc q2.c -o q2
ashborn@DESKTOP-I4G5Q4N:~/week10_p$ ./q2
File size      : 1 MB
Repetitions    : 50

Traditional I/O : 0.018283 seconds
Memory-mapped I/O: 0.007100 seconds

Memory-mapped I/O avoids repeated read()/write() system calls
and provides direct access to the file through memory.
```

OBSERVATION

The program successfully performed file reading and writing using both traditional `read()/write()` and `mmap()`. Memory-mapped I/O provides direct memory access to the file and reduces repeated system-call overhead, while traditional I/O is simpler to understand and implement.

RESULT

File reading and writing using `mmap()` were successfully implemented and compared with traditional `read()/write()` operations.